

Module 12 - chapter 19

Automate Provisioning EKS cluster with Terraform - Part 1

The project used in the following three lectures can be found here:

<https://gitlab.com/twn-devops-bootcamp/latest/12-terraform/terraform-learn/-/tree/feature/eks>

Introduction

Previously

- ▶ AWS Management Console (UI)
- ▶ eksctl

```
eksctl create cluster
```



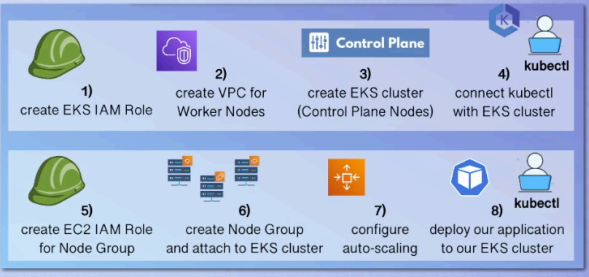
We learned all of the configuration need it

Introduction

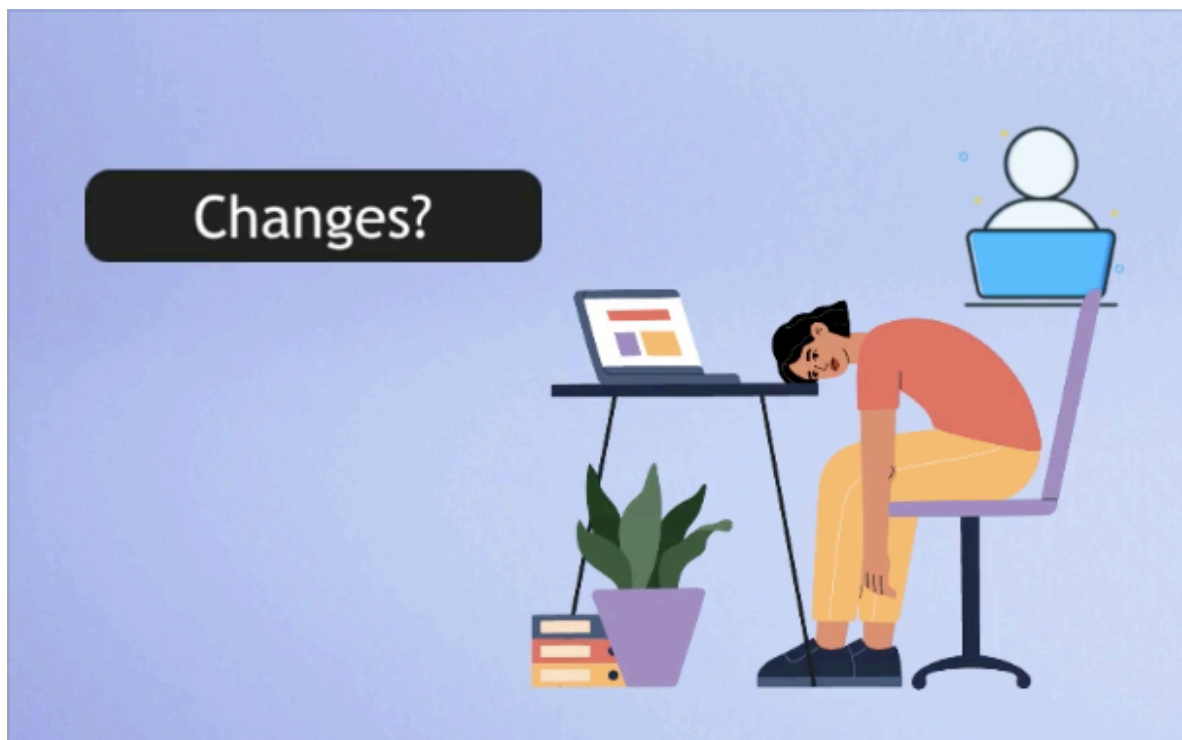
Previously

Many components to create and configure

Many components created in the background

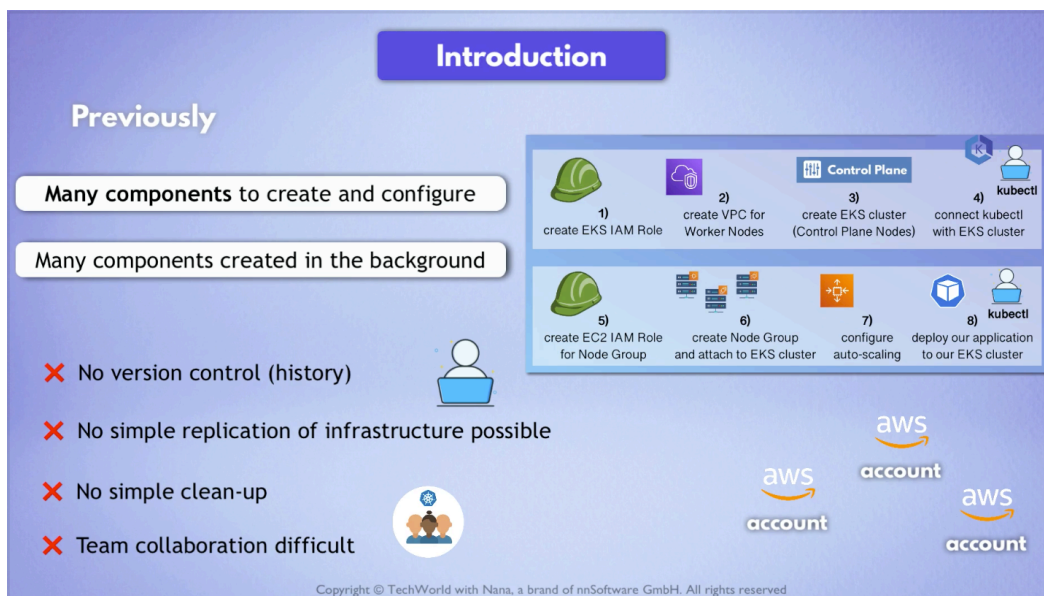


If we need to make changes, we will need to go to the vpc service or the worker node service or wherever the change is need it but we won't be able to track the changes and have version control



If we need to replicate the configuration in another AWS account it won't be easy to replicate.

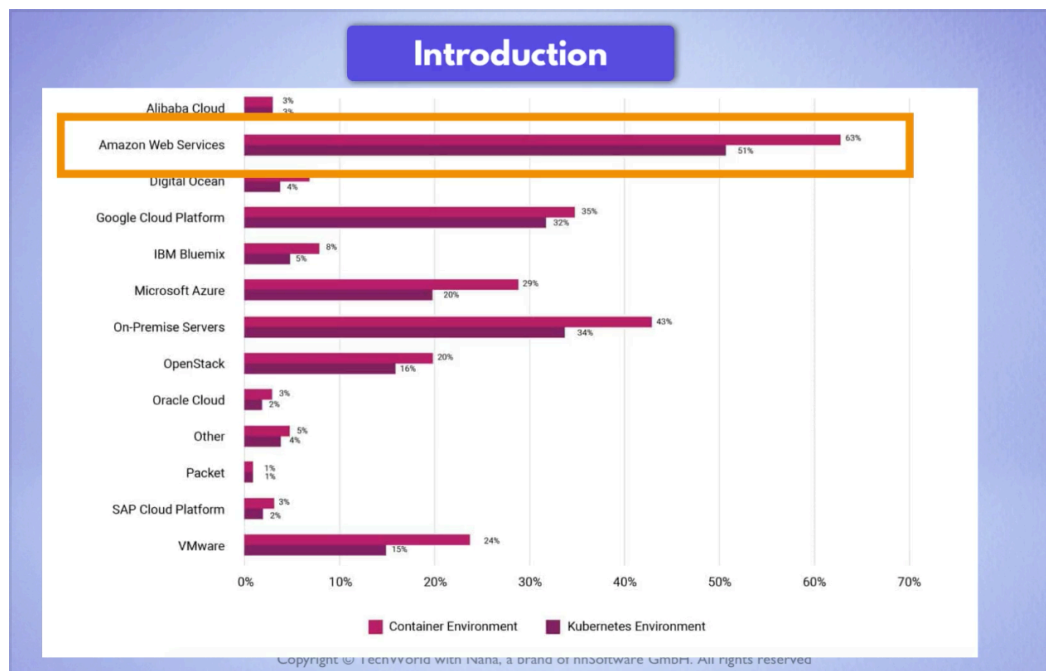
And if we try to clean up the whole infra it is also more complicated as we need to remember what we have done



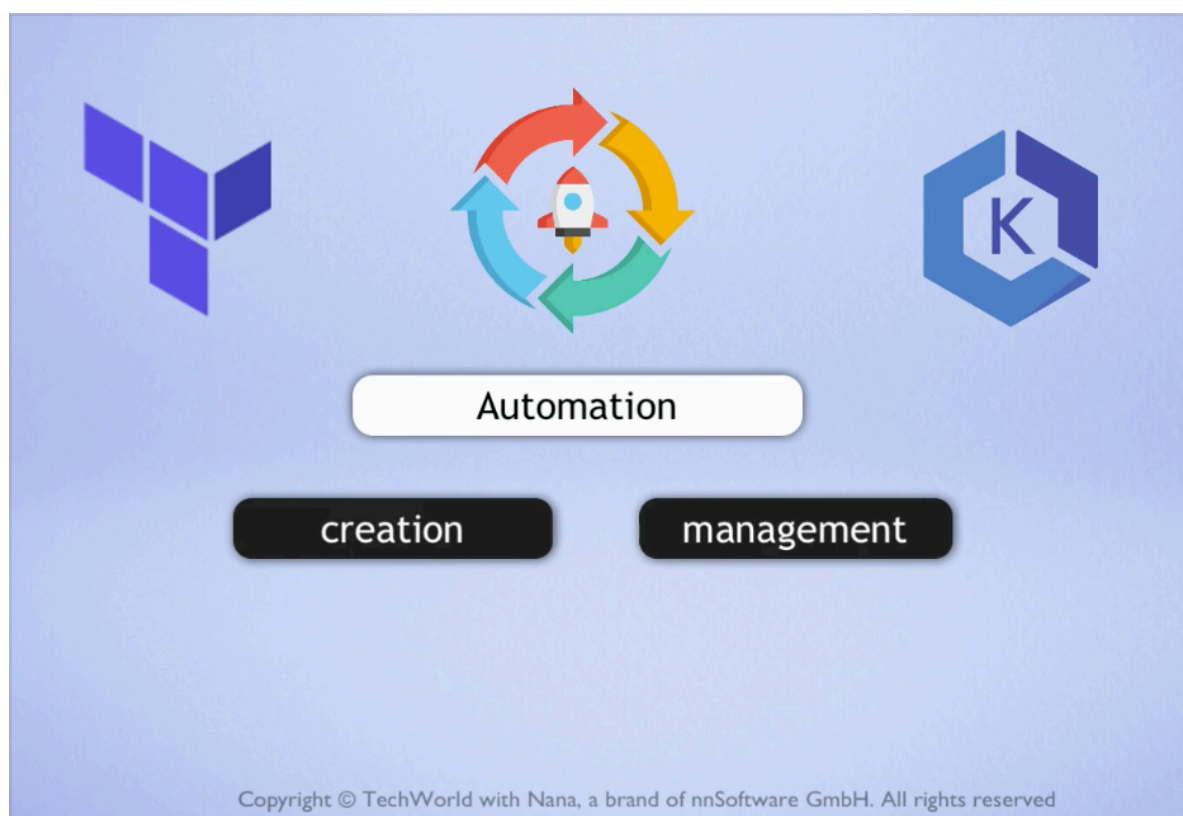
Also, sharing knowledge with colleagues when we created the infra by ourselves needs comprehensive documentation.

The process is inefficient.

More than half of k8s clusters run in AWS



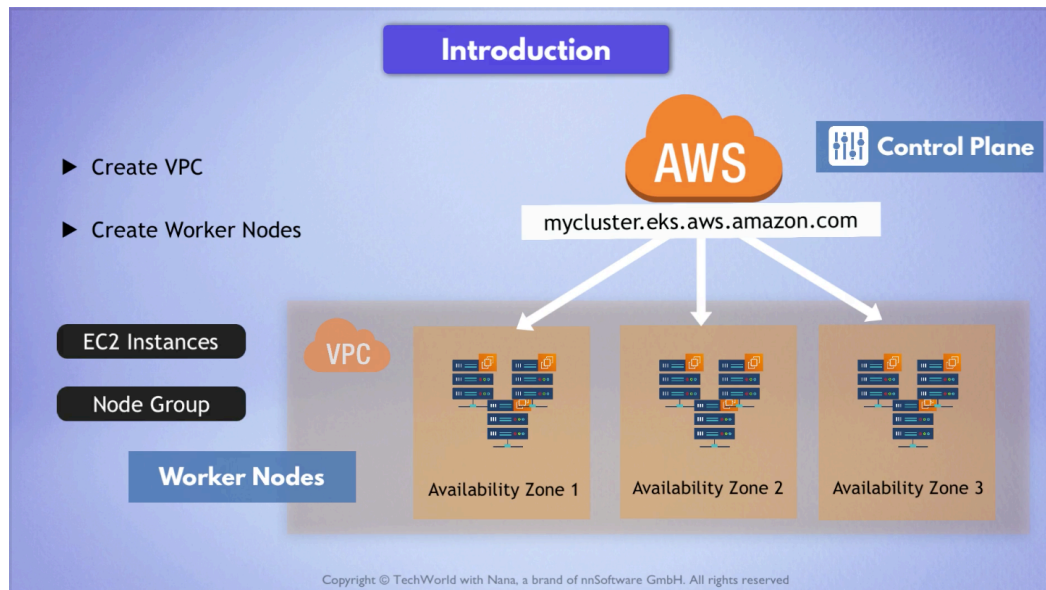
It makes sense to automate this process using terraform, AWS is power but also a complex platform to be dealing with manually.



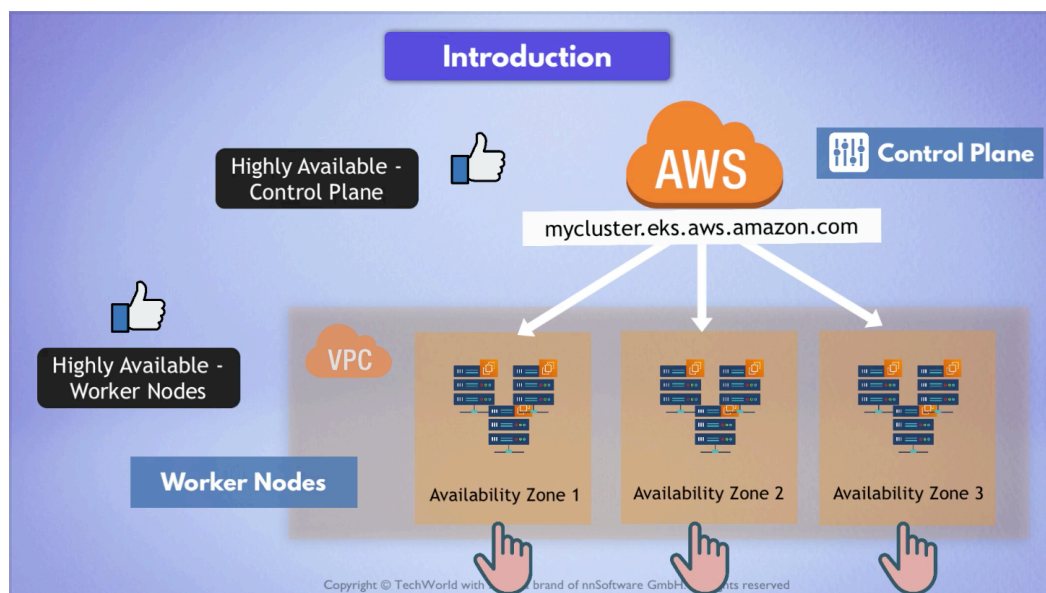
Terraform is good for creating and managing EKS clusters

Review of EKS components:

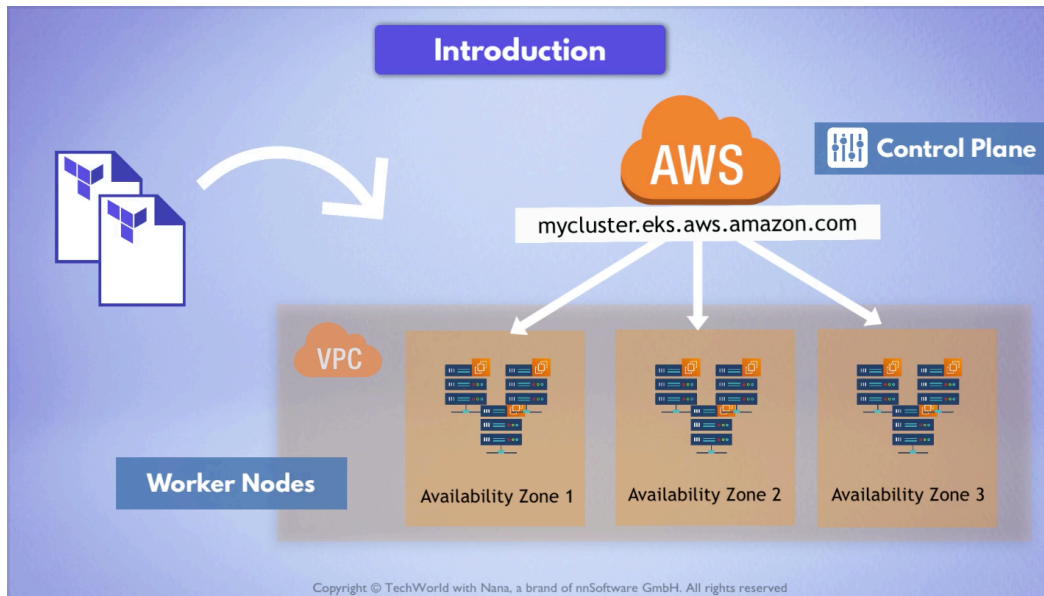
- Control plane -> managed by AWS
- Worker nodes (we need a VPC, worker Nodes inside the VPC which are EC2 instances that can be created directly or using Node group)



Also the cluster is created in a specific region with multiple AZs

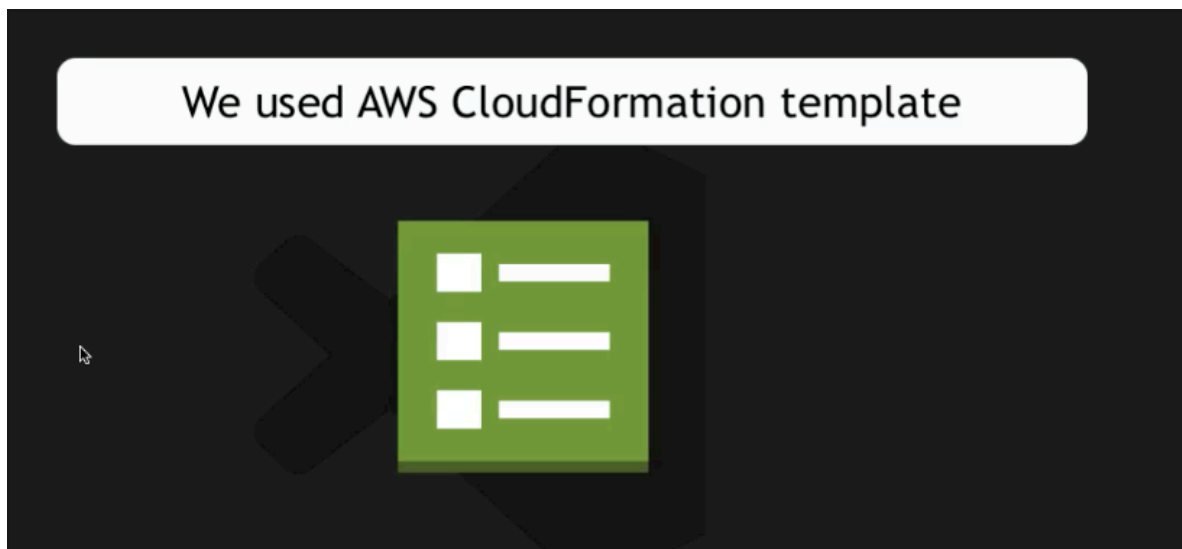


We will follow the steps using Terraform



VPC

We created the VPC using CloudFormation, because it has the configuration to create a VPC in the way the EKS cluster needs it



And chapter 12 we have created a VPC but not a VPC specific for EKS.

CloudFormation is an alternative for terraform specific for AWS

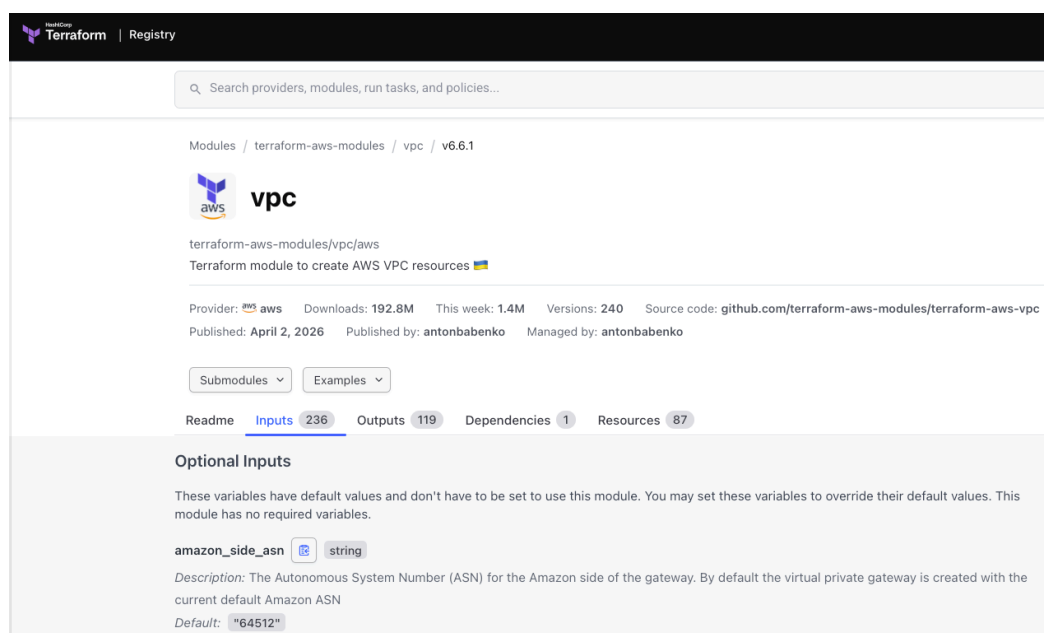


- We need something like CloudFormation template
- Alternative to Terraform
- CloudFormation is specific to AWS

we need something like cloudFormation template

-> we are going to use an existing ready terraform **module** to create the VPC for the EKS cluster

<https://registry.terraform.io/modules/terraform-aws-modules/vpc/aws/latest?tab=inputs>



The screenshot shows the Terraform Registry page for the `terraform-aws-modules/vpc` module. The page includes a search bar, the module name `vpc`, and the provider `aws`. It also displays statistics such as 192.8M downloads and 240 versions. The `Optional Inputs` section is visible, showing the `amazon_side_asn` variable with a default value of `"64512"`.

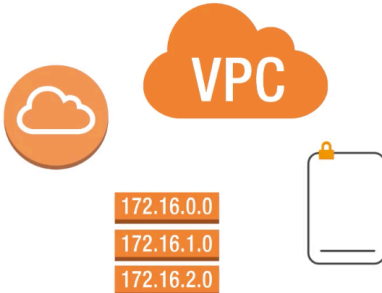
And in the background the module will create the resources needed (including gateway)

registry.terraform.io/modules/terraform-aws-modules/vpc/aws/latest/resources

This list contains all the resources this module plus any submodules may create. When using this module, it may create fewer resources if you use a submodule.

This module defines 82 resources.

- aws_cloudwatch_log_group.flow_log
- aws_customer_gateway.this
- aws_db_subnet_group.database
- aws_default_network_acl.this
- aws_default_route_table.default
- aws_default_security_group.this
- aws_default_vpc.this
- aws_egress_only_internet_gateway.this
- aws_eip.nat
- aws_elasticache_subnet_group.elasticache
- aws_flow_log.this
- aws_iam_policy.vpc_flow_log_cloudwatch
- aws_iam_role.vpc_flow_log_cloudwatch
- aws_iam_role_policy_attachment.vpc_flow_log_cloudwatch
- aws_internet_gateway.this
- aws_nat_gateway.this
- aws_network_acl.database



TECHWORLD WITH NANA

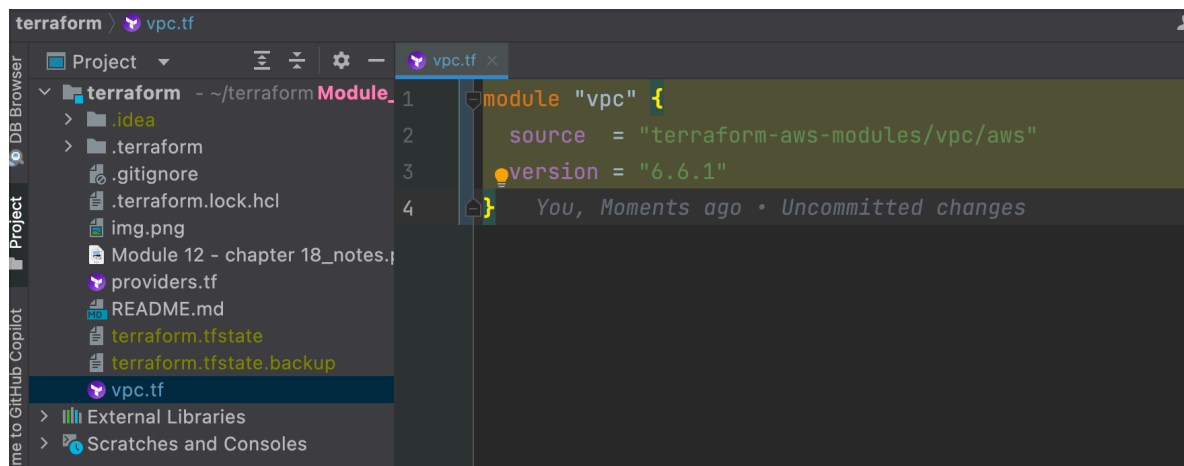
So we will copy the basic syntax from the documentation

Provision Instructions

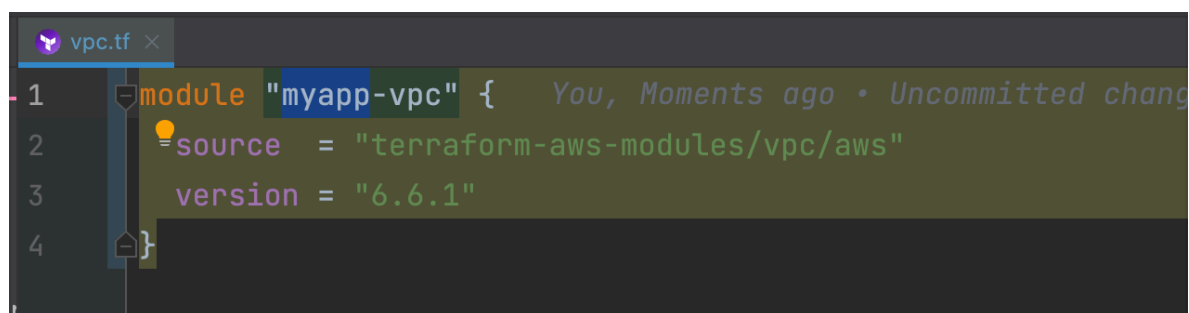
Copy and paste into your Terraform configuration, insert the variables, and run **terraform init** :

```
module "vpc" {  
  source = "terraform-aws-  
modules/vpc/aws"  
  version = "6.6.1"  
}
```

Copy configuration 



Let's edit the name of the module to "myapp-vpc"

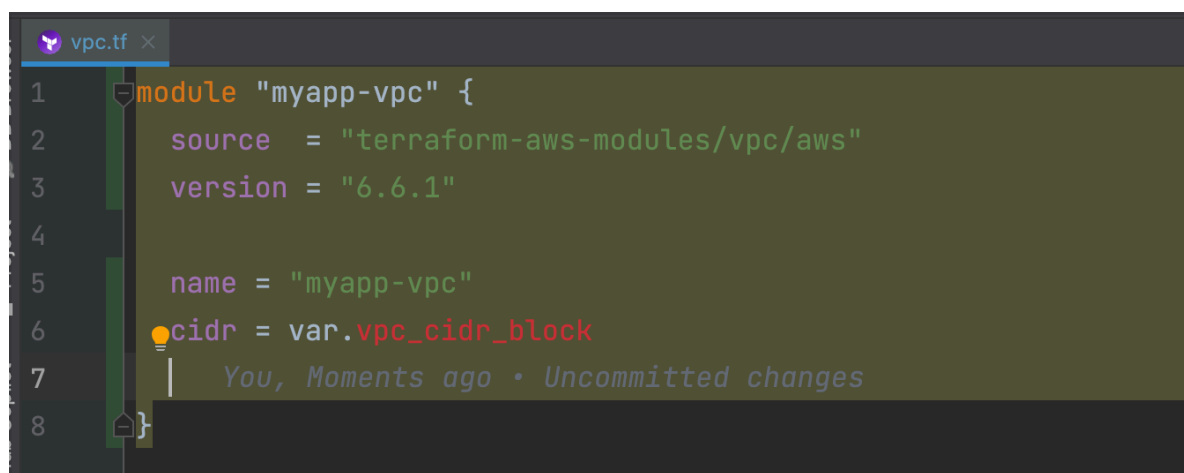


Also we need to download the Module code on 'terraform init' like we did with providers, so we need to reference where to download the module via the attribute 'source' 🙌

Now we pass the attributes that we need to create the VPC for the EKS cluster.

First one is the 'name' of the VPC inside AWS and we call it 'myapp-vpc' as well.

Then we specify the VPC cidr block



Next the subnet's cidr blocks

Best practice for subnets, to create for each AZ

- One public subnet
- One private subnet

The screenshot shows a Terraform code editor with a file named `vpc.tf`. The code defines a module `"myapp-vpc"` with the following configuration:

```
1 module "myapp-vpc" {
2   source = "terraform-aws-modules/vpc/aws"
3   version = "6.0.1"
4
5   name = "myapp-vpc"
6   cidr = var.vpc_cidr_block
7 }
8
```

Overlaid on the code is a white box with the text: **Best practice:**
1 private and 1 public subnet in each AZ

Below this box is a diagram illustrating the best practice across three Availability Zones (AZ 1, AZ 2, and AZ 3). Each AZ is represented by a light orange box containing two red rounded rectangles: one labeled "Public Subnet" and one labeled "Private Subnet".

We know our region has 3 AZ, so I need 3 private and 3 public subnets in total.

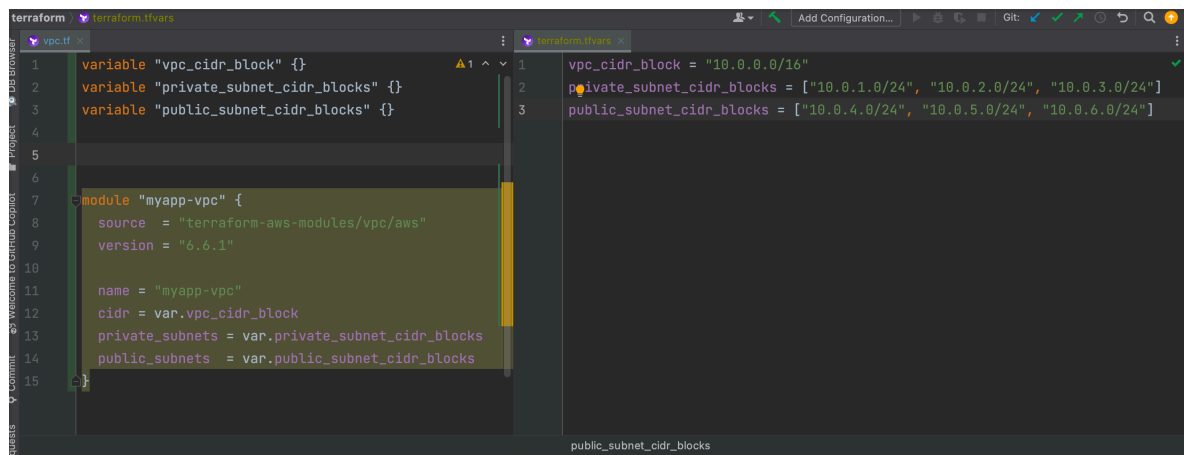
The screenshot shows the same Terraform code editor with the `vpc.tf` file. The code has been updated to:

```
1 module "myapp-vpc" {
2   source = "terraform-aws-modules/vpc/aws"
3   version = "6.6.1"
4
5   name = "myapp-vpc"
6   cidr = var.vpc_cidr_block
7   private_subnets = var.private_subnet_cidr_blocks
8   public_subnets = var.public_subnet_cidr_blocks
9 }
```

At the bottom of the editor, a status bar indicates: *You, Moments ago • Uncommitted changes*

Lines 7 will be an array of cidr block, same for line 8 🙏

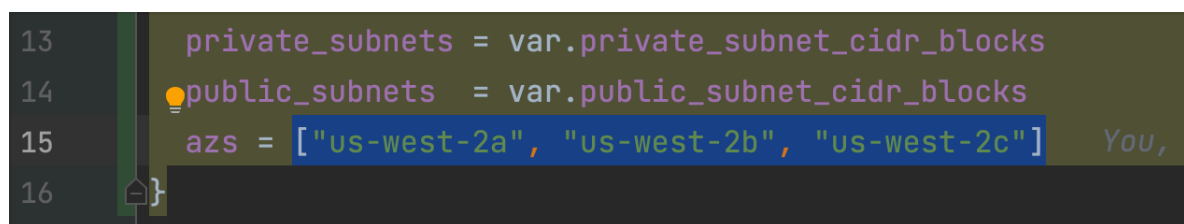
And we then define the variables and its values



```
1 variable "vpc_cidr_block" {}
2 variable "private_subnet_cidr_blocks" {}
3 variable "public_subnet_cidr_blocks" {}
4
5
6
7 module "myapp-vpc" {
8   source = "terraform-aws-modules/vpc/aws"
9   version = "6.6.1"
10
11   name = "myapp-vpc"
12   cidr = var.vpc_cidr_block
13   private_subnets = var.private_subnet_cidr_blocks
14   public_subnets = var.public_subnet_cidr_blocks
15 }
```

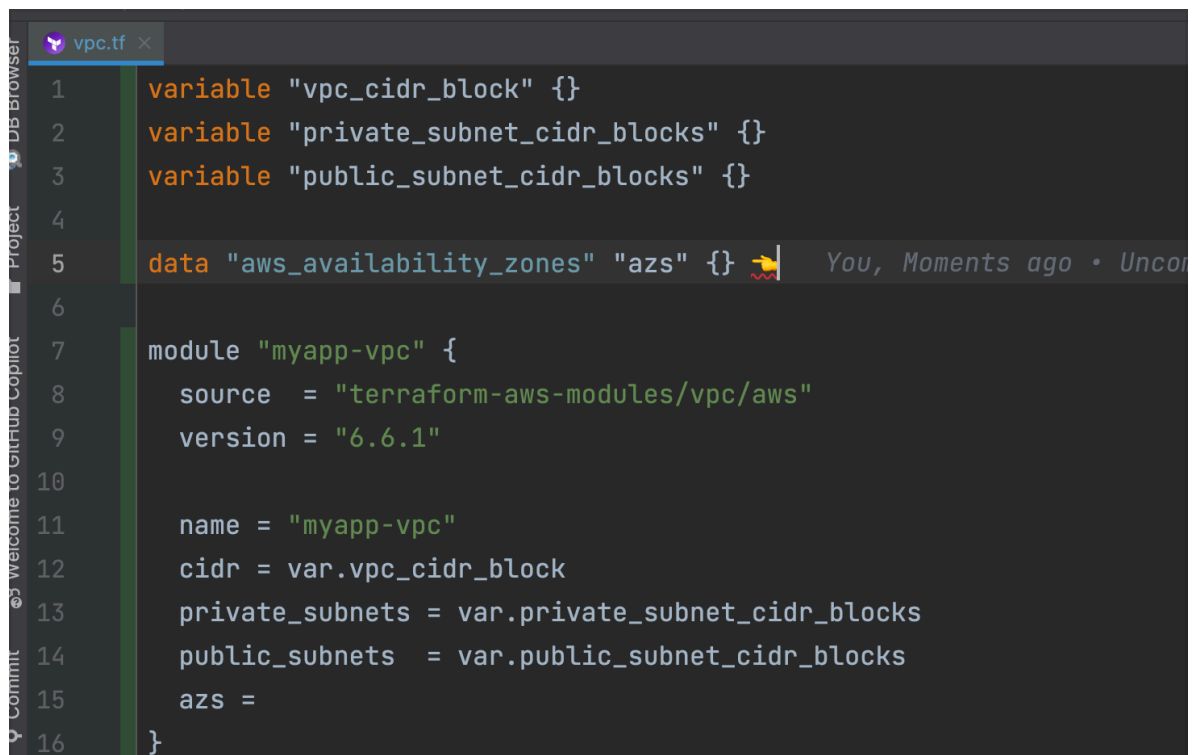
```
1 vpc_cidr_block = "10.0.0.0/16"
2 private_subnet_cidr_blocks = ["10.0.1.0/24", "10.0.2.0/24", "10.0.3.0/24"]
3 public_subnet_cidr_blocks = ["10.0.4.0/24", "10.0.5.0/24", "10.0.6.0/24"]
```

We need to define that those subnets need to be deployed in ALL the AZs.



```
13 private_subnets = var.private_subnet_cidr_blocks
14 public_subnets = var.public_subnet_cidr_blocks
15 azs = ["us-west-2a", "us-west-2b", "us-west-2c"]
16 }
```

It is better this et the AZs dynamically, so we will query AWS to get the AZ's of the region we want out VPC to have and their names:



```
1 variable "vpc_cidr_block" {}
2 variable "private_subnet_cidr_blocks" {}
3 variable "public_subnet_cidr_blocks" {}
4
5 data "aws_availability_zones" "azs" {}
6
7 module "myapp-vpc" {
8   source = "terraform-aws-modules/vpc/aws"
9   version = "6.6.1"
10
11   name = "myapp-vpc"
12   cidr = var.vpc_cidr_block
13   private_subnets = var.private_subnet_cidr_blocks
14   public_subnets = var.public_subnet_cidr_blocks
15   azs =
16 }
```

But to call "aws_availability_zones" in AWS I need to talk to AWS so I need the AWS provider and define the Region

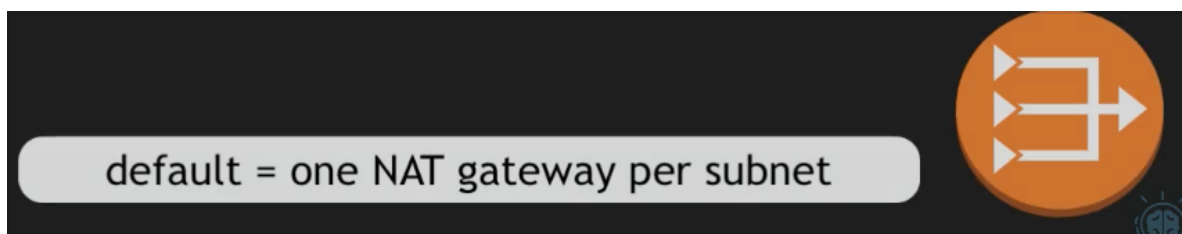
```
terraform providers.tf
1 provider "aws" {
2   region = "eu-west-2"
3 }
4
5 variable "vpc_cidr_block" {}
6 variable "private_subnet_cidr_blocks" {}
7 variable "public_subnet_cidr_blocks" {}
8
9 data "aws_availability_zones" "azs" {}
10
11 module "myapp-vpc" {
12   source = "terraform-aws-modules/vpc/aws"
13   version = "6.6.1"
14
15   name = "myapp-vpc"
16   cidr = var.vpc_cidr_block
17   private_subnets = var.private_subnet_cidr_blocks
18   public_subnets = var.public_subnet_cidr_blocks
19   azs =
20 }

providers.tf
1 terraform {
2   required_providers {
3     aws = {
4       source = "hashicorp/aws"
5       version = "~> 6.0"
6     }
7   }
8 }
```

Now I can use the data query as reference and select the AZs names

```
vpc.tf
1 provider "aws" {
2   region = "eu-west-2"
3 }
4
5 variable "vpc_cidr_block" {}
6 variable "private_subnet_cidr_blocks" {}
7 variable "public_subnet_cidr_blocks" {}
8
9 data "aws_availability_zones" "azs" {}
10
11 module "myapp-vpc" {
12   source = "terraform-aws-modules/vpc/aws"
13   version = "6.6.1"
14
15   name = "myapp-vpc"
16   cidr = var.vpc_cidr_block
17   private_subnets = var.private_subnet_cidr_blocks
18   public_subnets = var.public_subnet_cidr_blocks
19   azs = data.aws_availability_zones.azs.names
20 }
```

Now we will set the nat gateway for transparency even though it isn't need it:



And also we will set 'single_nat_gateway' to true to have a single shared gateway for all the private subnets

All private subnets will **route** their internet traffic through this single NAT gateway

```
20
21     enable_nat_gateway = true
22     single_nat_gateway = true
```

And enable 'enable_dns_hostnames' inside the VPC

```
21     enable_nat_gateway = true
22     single_nat_gateway = true
23     enable_dns_hostnames = true
24 }
```

For example when an EC2 instance gets created it will get assigned the public IP address, the private IP address, and also get assigned the public and private dns names.

So AWS automatically assigns a DNS name to every EC2 instance like:

-> ec2-18-185-32-41.eu-west-1.compute.amazonaws.com

This is just AWS mapping that ugly hostname to the instance's public IP internally. It's useful so that other AWS services (like EKS nodes talking to each other) can resolve instances by name instead of hardcoded IPs.

Finally, we add a bunch of tags

This is the first one -> "kubernetes.io/cluster/myapp-eks-cluster" = "shared"

```
24
25     tags = {
26       "kubernetes.io/cluster/myapp-eks-cluster" = "shared"
27     }
28 }
```

AWS has a component called cloud control manager -> c-c-m

And the tags help c-c-m to identify which VPC and subnets it should connect to.

So I need the same tags for the public and private subnets as well

```
25     tags = {
26         "kubernetes.io/cluster/myapp-eks-cluster" = "shared"
27     }
28
29     public_subnet_tags = {
30         "kubernetes.io/cluster/myapp-eks-cluster" = "shared"
31     }
32
33     private_subnet_tags = {
34         "kubernetes.io/cluster/myapp-eks-cluster" = "shared"
35     }
36
37 }
```

Q:

is this c-c-m the same as the control manager of the control plane nodes?

A:

● No, they are different things — easy to confuse though:

kube-controller-manager (control plane)

- Part of the Kubernetes control plane
- Handles generic Kubernetes logic: pod lifecycle, replication controllers, node health, deployments rolling out, etc.
- Cloud-agnostic — knows nothing about AWS, Azure, GCP

Cloud Controller Manager / CCM (cloud-specific)

- A separate component, specific to your cloud provider
- Handles the bridge between Kubernetes and AWS specifically: load balancers, routes, node registration
- On EKS, AWS runs this for you as part of the managed control plane

Think of it this way:

- kube-controller-manager — "make sure 3 replicas of this pod are running"
- CCM — "go create an AWS Load Balancer for this Kubernetes Service"

They run separately and have different responsibilities. The CCM was actually split out of kube-controller-manager years ago precisely to keep cloud-specific logic out of the core Kubernetes codebase.

What is the CCM?

The Cloud Controller Manager is a Kubernetes component that runs inside your EKS cluster and talks to AWS on your behalf. Its main job is things like: "a developer just created a Kubernetes Service of type LoadBalancer — go create an AWS Load Balancer for it automatically."

The problem it solves with tags

AWS doesn't automatically know which VPCs and subnets belong to which EKS cluster. The CCM uses these tags to discover them. Without the tags, the CCM can't find where to create load balancers, assign IPs, etc.

What the tag means:

"kubernetes.io/cluster/myapp-eks-cluster" = "shared"

- The key identifies which cluster (myapp-eks-cluster is your cluster name)
- "shared" means this resource can be used by multiple clusters. The alternative is "owned" — only one cluster uses it

Tags - Use Cases



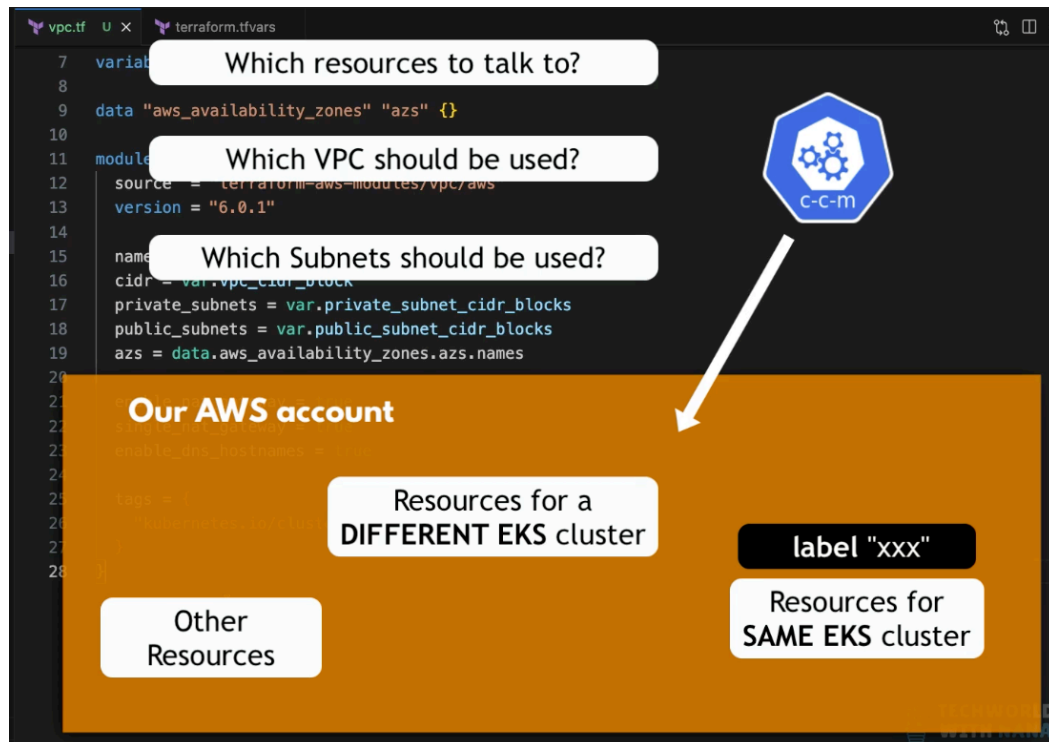
- For human consumption to have more information
- Label for referencing components from other components (programmatically)
- E.g. for Cloud Controller Manager

So when we create an EKS cluster, AWS creates a process in the control plane called Kubernetes cloud controller manager - ccm.

Ccm is the one the orchestrates connecting to the VPCs, connecting to the subnets, connecting with the worker nodes and all this configurations -> it talks to the resources in our AWS account (and also creating some other things).

So Kubernetes ccm needs to know which resources in our account it should talk to, it needs to know which VPC should be used in a cluster, which subnet should be used in the cluster and so on. Because we may have multiple VPCs and multiple subnets we need to tell the control plane or AWS (AWS via the control plane) -> use these VPCs and these subnets for these specific cluster.

We also may have, multiple VPCs for multiple EKS -> so it has to be a specific label that ccm can actually detect and identify.



We also need to add couple of more tags to the subnets related to elastic load balancer - elb:

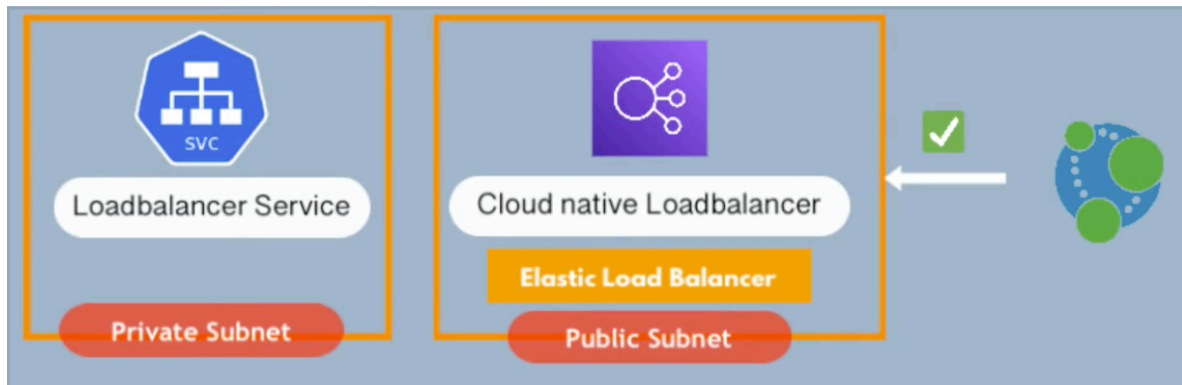
- > "kubernetes.io/role/elb" = "1" # internet-facing LB
- > "kubernetes.io/role/internal-elb" = "1" # internal L

```

27  }
28  }
29  public_subnet_tags = {
30    "kubernetes.io/cluster/myapp-eks-cluster" = "shared"
31    "kubernetes.io/role/elb" = 1
32  }
33  }
34  private_subnet_tags = {
35    "kubernetes.io/cluster/myapp-eks-cluster" = "shared"
36    "kubernetes.io/role/internal-elb" = 1
37  }
38  }
39  }

```

When we create a load balancer service in Kubernetes -> then k8s will provision a cloud native load balancer for that service



And the cloud load balancer will be provisioned in the public subnet to become the entry point to the cluster (accessible from outside) and this cloud load balancer gets an external IP address so we can communicate to it from outside like from browser requests or from other clients.

Since we have subnets and private subnets in our VPC, the public subnet allows communication with internet. The private subnet is closed to internet.

So k8s needs to know which one is the public subnet so that it can create and provision that load balancer in the public subnet so that the load balancer can be accessed from the internet.

And the internal load balancer will be created inside the private subnets.

We just need to make sure the load balancers are created in the correct subnets.

Without elb and internal-elb tags, the CCM won't know which subnets to use when creating load balancers.

Your notes	What's actually doing it
"k8s will provision a cloud native load balancer"	CCM detects the Service and calls AWS API
"k8s needs to know which subnet is public"	CCM reads the <code>kubernetes.io/role/elb</code> tag
"internal load balancer in private subnet"	CCM reads the <code>kubernetes.io/role/internal-elb</code> tag

```

16 cidr = var.vpc_cidr_block
17 private_subnet_cidr_blocks = [cidrsubnet(cidr, 8, 1),
18     cidrsubnet(cidr, 8, 2)]
19 azs = data.aws_availability_zones.azs.names
20
21 enable_nat_gateway = true
22 single_nat_gateway = true
23 enable_dns_hostnames = true
24
25 tags = {
26     "kubernetes.io/cluster/myapp-eks-cluster" = "shared"
27 }
28
29 public_subnet_tags = {
30     "kubernetes.io/cluster/myapp-eks-cluster" = "shared"
31     "kubernetes.io/role/elb" = 1
32 }
33
34 private_subnet_tags = {
35     "kubernetes.io/cluster/myapp-eks-cluster" = "shared"
36     "kubernetes.io/role/internal-elb" = 1
37 }
38 }

```

These tags are required!

Now we can check the VPC configuration, but first because we created a module we need to run
-> terraform init

```

Initializing modules...2/Terraform_chapter-19 +13 !1 terraform init
Initializing provider plugins found in the configuration...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.50.0

Initializing the backend...

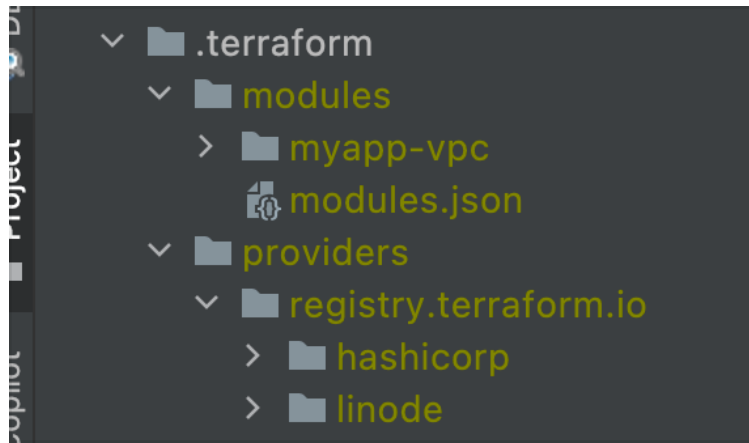
Initializing provider plugins found in the state...

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

```

Here we can see that the module and the provider got downloaded in the
'.terraform' folder



And then

-> terraform plan

```
}

Plan: 23 to add, 0 to change, 0 to destroy.
```

```
Note: You didn't use the -out option to save this
```



```

# module.myapp-vpc.aws_subnet.public[2] will be created
+ resource "aws_subnet" "public" {
  + arn                                     = (known after apply)
  + assign_ipv6_address_on_creation        = false
  + availability_zone                      = "eu-west-2c"
  + availability_zone_id                   = (known after apply)
  + cidr_block                             = "10.0.6.0/24"
  + enable_dns64                           = false
  + enable_resource_name_dns_a_record_on_launch = false
  + enable_resource_name_dns_aaaa_record_on_launch = false
  + id                                     = (known after apply)
  + ipv6_cidr_block                        = (known after apply)
  + ipv6_cidr_block_association_id         = (known after apply)
  + ipv6_native                            = false
  + map_public_ip_on_launch                = false
  + owner_id                              = (known after apply)
  + private_dns_hostname_type_on_launch    = (known after apply)
  + region                                 = "eu-west-2"
  + tags                                   = {
    + "Name"                               = "myapp-vpc-public-eu-west-2c"
    + "kubernetes.io/cluster/myapp-eks-cluster" = "shared"
    + "kubernetes.io/role/elb"              = "1"
  }
  + tags_all                               = {

```

```

# module.myapp-vpc.aws_subnet.private[0] will be created
+ resource "aws_subnet" "private" {
  + arn                                     = (known after apply)
  + assign_ipv6_address_on_creation        = false
  + availability_zone                      = "eu-west-2a"
  + availability_zone_id                   = (known after apply)
  + cidr_block                             = "10.0.1.0/24"
  + enable_dns64                           = false
  + enable_resource_name_dns_a_record_on_launch = false
  + enable_resource_name_dns_aaaa_record_on_launch = false
  + id                                     = (known after apply)
  + ipv6_cidr_block                        = (known after apply)
  + ipv6_cidr_block_association_id         = (known after apply)
  + ipv6_native                            = false
  + map_public_ip_on_launch                = false
  + owner_id                              = (known after apply)
  + private_dns_hostname_type_on_launch    = (known after apply)
  + region                                 = "eu-west-2"
  + tags                                   = {
    + "Name"                               = "myapp-vpc-private-eu-west-2a"
    + "kubernetes.io/cluster/myapp-eks-cluster" = "shared"
    + "kubernetes.io/role/internal-elb"        = "1"
  }
  + tags_all                               = {
    + "Name"                               = "myapp-vpc-private-eu-west-2a"
    + "kubernetes.io/cluster/myapp-eks-cluster" = "shared"
  }

```

```
# module.myapp-vpc.aws_route.private_nat_gateway[0] will be created
+ resource "aws_route" "private_nat_gateway" {
  + destination_cidr_block = "0.0.0.0/0"
  + id                     = (known after apply)
  + instance_id            = (known after apply)
  + instance_owner_id      = (known after apply)
  + nat_gateway_id         = (known after apply)
  + network_interface_id   = (known after apply)
  + origin                 = (known after apply)
  + region                 = "eu-west-2"
  + route_table_id         = (known after apply)
  + state                  = (known after apply)

  + timeouts {
    + create = "5m"
  }
}

# module.myapp-vpc.aws_route.public_internet_gateway[0] will be created
+ resource "aws_route" "public_internet_gateway" {
  + destination_cidr_block = "0.0.0.0/0"
  + gateway_id             = (known after apply)
  + id                     = (known after apply)
```

Questions

Q:

I have this notes: And enable 'enable_dns_hostnames' inside the VPC For example when an EC2 instance gets created it

will get assigned the public IP address, the private IP address, and also get assigned the public and private dns names, I am not sure about the dns names part can you explain it? don't we have to provide the dns name? I meand don't we have to buy a name first?

A:

There are two different things here:

AWS-assigned DNS names (automatic, free)

When enable_dns_hostnames = true on your VPC, AWS automatically assigns a DNS name to every EC2 instance. You don't buy or register anything. They look like:

ec2-18-185-32-41.eu-west-1.compute.amazonaws.com

This is just AWS mapping that ugly hostname to the instance's public IP internally. It's useful so that other AWS services (like EKS nodes talking to each other) can resolve instances by name instead of hardcoded IPs.

Custom domain names (what you're thinking of)

That's a separate thing — if you want `myapp.com` pointing to your EC2, yes, you'd need to buy/own that domain and configure it (e.g. via Route 53). But that's unrelated to `enable_dns_hostnames`.

For EKS specifically, `enable_dns_hostnames = true` is required because:

- EKS nodes need to resolve each other by hostname - The EKS control plane communicates with worker nodes using these AWS-assigned DNS names

So you don't need to buy anything — just set the flag and AWS handles the rest.